Polyglot Incident Response

The staff dashboard says resolved.
The resident page says open.

Day 1: trace the disagreement and test delivery behavior
Day 2: apply operational checks to your final project

An authoritative record and a projection

Term	Responsibility in this application
Authoritative ticket	Accepts the staff update that establishes ticket status
Resident projection	Stores a derived copy arranged for the resident page
Source version	Identifies the accepted order of updates for one ticket

PostgreSQL owns ticket status in this case.
MongoDB stores the derived resident view. This is a design choice.

Two separate writes have two outcomes

Application action	Possible observation
Commit the PostgreSQL update	Source accepts resolved, version 17
Send the MongoDB update	Connection fails or the client times out
Return an error to the browser	The committed PostgreSQL update still exists

Reversing the write order changes which partial result is possible.
One database transaction does not automatically include another service.

A timeout does not identify the stored outcome

What happened remotely	What the client can observe
The request never reached the server	No reply before the timeout
The server applied the write, but the reply was lost	No reply before the timeout

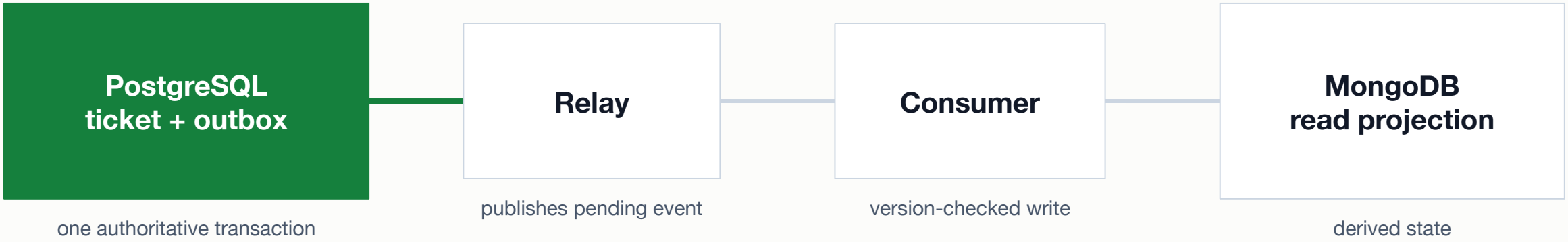
Read the relevant record and correlate the event ID.
A retry must be safe even if the first attempt already applied.

Ticket identity, event identity, and version

Field	Example	Meaning
ticket_id	1008	The same business ticket across stores
event_id	evt-1008-17	One particular recorded change
source_version	17	The source order for this ticket

Version 18 of ticket 1008 follows version 17 of that ticket.
It says nothing about version 18 of another ticket or a wall-clock time.

The transactional outbox



The outbox closes the source update/event gap; delivery, retries, lag, and reconciliation still remain.

The notebook makes the commit boundary visible

Notebook 08: Polyglot Incident

Part	What runs
Transaction demonstration	SQLite tables and actual local commits or rollbacks
Delivery experiment	One Python dictionary representing the resident copy
Reconciliation	Comparison of source IDs and owned field values

Choose Open in Colab on the weekly page. Run the cells in order.
No package installation, cloud database account, or database password is needed.

An exception rolls back the local transaction

```
try:
    with source_db:
        source_db.execute(
            "UPDATE tickets SET status = ?, source_version = ? "
            "WHERE ticket_id = ?",
            ("resolved", 17, 1008),
        )
        raise RuntimeError("Before the event insert")
except RuntimeError as error:
    print(error)
```

After fresh setup: ticket 1008 is open, version 16. Outbox rows: 0.

Section 1 excerpt. The exception leaves the transaction block before it is caught.

The ticket and its event commit together

```
with source_db:
    changed = source_db.execute(
        "UPDATE tickets SET status = ?, source_version = ? "
        "WHERE ticket_id = ? AND source_version = ?",
        ("resolved", 17, 1008, 16),
    )
    if changed.rowcount == 1:
        source_db.execute("INSERT INTO outbox VALUES (?, ?, ?, ?)",
                          ("evt-1008-17", 1008, "resolved", 17))
```

Successful-branch excerpt after setup. The complete notebook also reports a zero match. A failed event insert rolls back this transaction's ticket update.

Recorded events still need delivery

Notebook stage	Ticket 1008	Outbox rows
Fresh setup	open, version 16	0
Failure and rollback	open, version 16	0
Successful local commit	resolved, version 17	1
Later accepted close operation	closed, version 18	2

The paper incident observes version 17. Version 18 occurs later in the experiment. These local cells do not run a relay or consumer.

Idempotency and ordering are different properties

Operation repeated twice	Effect
Set status to resolved	Same final status as one application
Add one to a counter	Two increments rather than one
Set an older status after a newer status	Can regress even if the old operation is repeat-safe

A retry-safe state assignment still needs an ordering rule.
Our complete-state events carry a version assigned by the source.

A version guard rejects stale deliveries

```
for event in deliveries:
    if (ENFORCE_VERSION
        and event["source_version"] <= projection["source_version"]):
        print("Ignore old or duplicate event:", event["event_id"])
    else:
        projection["status"] = event["status"]
        projection["source_version"] = event["source_version"]
        projection["last_event_id"] = event["event_id"]
        print("Apply:", event["event_id"])
```

Section 2 excerpt. Each cell run resets the projection to open, version 16.
The dictionary loop models one consumer. It is not a concurrent database write.

The default delivery trace

Incoming version	Guarded action	Stored status and version
17	Apply	resolved, 17
17 again	Ignore duplicate	resolved, 17
16 late	Ignore older state	resolved, 17
18	Apply	closed, 18

All four deliveries describe ticket 1008. Each event contains its complete projected status, not an instruction to increment a counter.

Individual experiment: a delayed old event

```
Move version 16 to the end of deliveries.
Run with ENFORCE_VERSION = True.
Change it to False and rerun.
Restore True and rerun.
```

Compare the final status and version in the broken and repaired runs.
Use those observed results in your Brightspace incident update.

Complete state and changes require different handling

What the event means	If version 18 arrives before 17
The current total is 3	An authoritative complete state can supersede an older one
Add 2, after an earlier add 1	Skipping the earlier increment loses part of the total

The lab uses complete-state events. Delta events require a policy for ordering, deduplication, and reconstruction. This is an optional extension.

MongoDB can compare and update atomically

```
db.ticket_projection.updateOne(
  { _id: 1008, source_version: { $lt: 18 } },
  { $set: { status: "closed", source_version: 18,
            last_event_id: "evt-1008-18" } }
)
```

Reference for an existing practice document. MongoDB checks the filter and changes this document in one atomic operation.
The notebook explains this command but does not execute it.

Zero matches has several possible explanations

Existing state for _id 1008	Why version < 18 does not match
Already version 18 or newer	The normal retry has nothing newer to apply
Document is absent	There is no record to update
Version 18 with the wrong status	The version matches, but reconciliation must detect the wrong value

A version-filtered upsert can collide with an existing newer _id.
Initialization and repair of corrupted values need explicit policies.

The supplied incident follows one event

Time (UTC)	Observation
14:03:12	PostgreSQL commits resolved, version 17, and evt-1008-17
14:03:13	The relay marks the event published; the consumer records receipt
14:03:13-14	The consumer times out writing and queues a retry
14:04:00	The retry worker pauses with an expired database credential
14:04:05	The resident projection still contains open, version 16

The later read confirms this projection is behind.
The record does not establish the state of every other ticket.

Repair follows the authoritative direction

Restore the consumer's approved access.
Resume the recorded work through the version-checked handler.
Compare the resulting projection with the current source.

Then inspect other queued tickets or consumer lag.
Keep PostgreSQL's accepted status. Reverting it to match a stale page would discard the staff's valid update.

Equal counts can conceal three different problems

Ticket ID	Source status / version	Resident status / version
1008	closed / 18	closed / 18
1009	resolved / 5	absent
1010	resolved / 7	open / 7
9999	absent	open / 1

Three source records. Three resident records. Three discrepancies.
Compare the union of IDs and the source-owned fields.

A separate candidate preserves the repair direction

```
rebuilt_records = {
    ticket_id: dict(fields)
    for ticket_id, fields in source_records.items()
}
print(rebuilt_records.keys() == source_records.keys())
print(rebuilt_records == source_records)
```

Both comparisons return True. The source stays unchanged.

A live rebuild also needs to preserve separately owned fields, account for new writes, and switch readers safely. This model does not perform that cutover.

Lab submission: one incident update

Write one Brightspace text response explaining the user impact, where progress stopped, and your proposed repair.
Include the final status and version from your broken and repaired runs.
Name one broader check before closing the incident.

Work individually. The notebook is your working material.
No notebook attachment or separate timeline is required.

Day 2: Final Project Operations Clinic

A useful query runs.
The next task is to make its operation understandable.

Use the model, queries, and project guide you already have.
The final-project assignment remains the requirements source.

Access tests depend on the identity

Test	What a successful result establishes
Administrator reads a record	That administrator has access
Intended reader reads an allowed record	The chosen read path works for that identity
Intended reader attempts a forbidden write	An expected denial tests the stated permission boundary
Invalid value fails a constraint	The integrity rule rejects that value, not who may access it

Use disposable project data and the intended identity.
Keep passwords, connection strings, and privileged service keys out of the demo.

An index decision begins with one query

Example workload: find a resident's recent open tickets.

Keep fixed	Compare before and after
Filter, sort, dataset, and returned rows	The selected plan and examined work
The exact query parameters	Which index fields support the filter and order
The environment and measurement method	Any observed timing, with its limits

A small collection can legitimately use a scan. Record what changed and the cost of maintaining the index on writes.

A restore test answers more than a backup command

Observation in an isolated restore target	What it checks
Expected IDs and selected field values	Data survived in the expected form
A familiar query returns its expected answer	The restored data supports that operation
Indexes, constraints, and access are inspected	Required non-data behavior exists or is restored separately
The restore command and elapsed time are recorded	The procedure ran under those stated conditions

Use the recovery approach appropriate to your platform and artifact.
A CSV or JSON export alone does not recreate every database object.

Two stores need an explicit recovery order

For a rebuildable resident projection:
restore the authoritative source in isolation,
reconstruct the source-owned projection fields,
compare IDs and values, then coordinate access.

A MongoDB collection with independent historical facts needs its own recovery source. Two dumps from different times do not automatically represent one application-consistent point.

A demonstration another person can follow

Show	Explain
One query with its input and result	The question, record meaning, and expected answer
One operational decision	What you tested and what changed
A failure or limitation	How you detected it and what remains uncertain
A saved result if the cloud connection fails	When and where it ran, without claiming it is a live result

The notebook or script can be the presentation.
A second slide deck is optional in the final-project assignment.

Individual clinic work

Open the final-project assignment and your existing package.
Choose the highest-risk unfinished item in the rubric.
Complete and check that item in the same notebook, code, or guide.

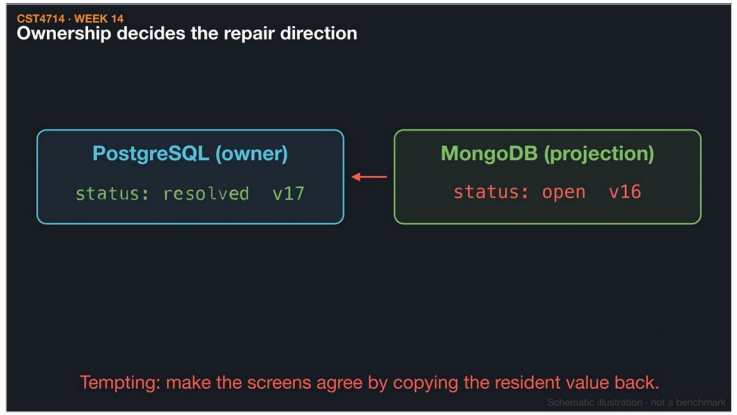
Possible work: a query correction, a permission test, an index comparison, or a separate restore check. A one-platform project remains a valid choice. There is no separate Week 14 project submission.

Explaining a database incident in an interview

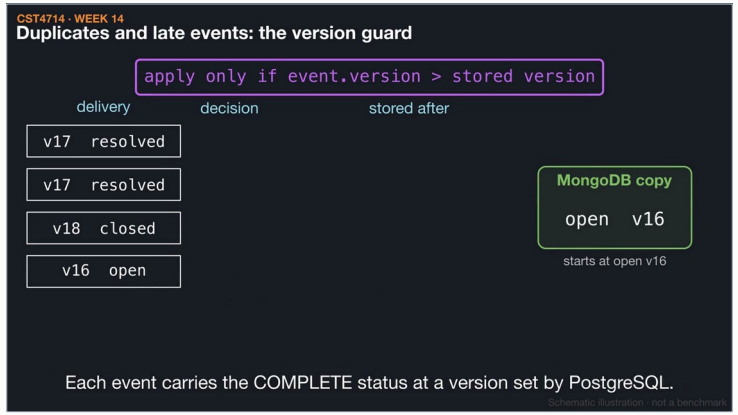
In a class experiment, I traced a stale resident view to a paused consumer. I tested duplicate and delayed full-state events, then compared IDs and values across the source and projection. The local model showed why a version guard matters and why matching record counts alone can miss corruption.

Describe the work you actually ran. The local notebook does not establish experience operating a distributed production service.
Next week: course review, final demonstrations, and career explanations.

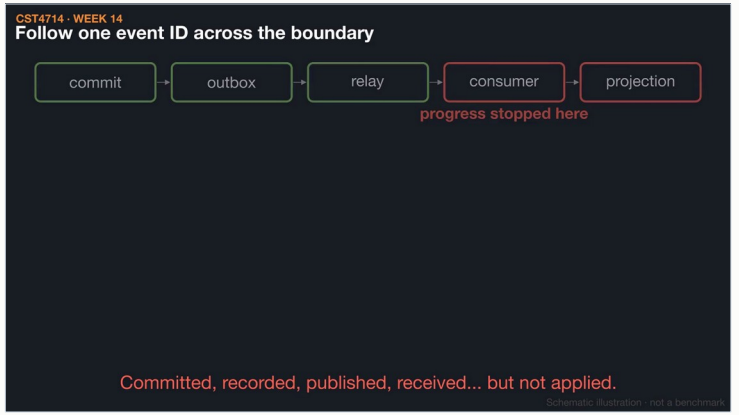
This week's videos



Polyglot persistence: one owner per fact (1:30)



Keeping two databases consistent: outbox and version guard (1:42)



Triage a cross-database incident (1:24)

In PowerPoint Slide Show, click a thumbnail to play full screen. Embedded videos work offline.